METRO HACKS

13 Problem Statements Deep Research & Solution Map

A practical guide to problem interpretation, prototype architecture, related GitHub projects, open data and feasibility.

Research scope

- All 13 problem statements in the Metro Hacks Participant Brief, with their original PS IDs and offline requirements.
- Existing public repositories, standards and APIs that can be reused as complete prototypes or technical building blocks.
- Practical two-hour scope, data availability, limitations and validation requirements.
- Research date: 10 October 2026. Repository pages and documentation were reviewed online; code was not cloned or executed for this report.

The main finding

There are reusable open-source components for most of these problems, but only a few projects closely match the exact brief. For a credible hackathon demo, reuse a proven component, build one end-to-end workflow, and label simulated data and unvalidated predictions honestly.

At a glance

The participant brief frames this as a two-hour prototype challenge, warns that connectivity can drop, and states that no datasets will be provided. Three problems explicitly require offline operation. Download libraries, map tiles, models and sample data before starting.

ID / problem	Track / offline	Core prototype output
PS-01 Winning Back the Empty Seats	Metro Ops / No	Segments + interventions
PS-02 Fix It Before It Breaks	Metro Ops / No	Weekly asset-risk roster
PS-03 The Technician in the Tunnel	Metro Ops / Required	Offline cited manual answer
PS-04 No Day Is an Average Day	Metro Ops / No	Headway recommendations
PS-05 The Stadium Surge	Metro Ops / No	Timed crowd plan
PS-06 Footfall to Revenue	Metro Ops / No	Ad/retail rate card
PS-07 Every Complaint Heard	Metro Ops / Required	Offline complaint triage
PS-08 One City, One Journey	City Public Transport / No	Multimodal itinerary/fare
PS-09 Clear the Exit	City Public Transport / Required	Offline photo + hotspot map
PS-10 Diversions Without Disruption	City Public Transport / No	Diversion map/messaging
PS-11 Beat the Heat	City Public Transport / No	Shaded walking routes
PS-12 Monsoon Mode	City Public Transport / No	24-hour readiness plan
PS-13 The Two-Minute Morning Brief	City Public Transport / No	Daily MIS one-pager

How to read the matches

Some links are direct application references; others are reusable models, standards or engines. Every repository still needs a compatibility check, local-data validation and a real end-to-end demo.

Ahmedabad data reality check

AMTS/BRTS: The community amd-gtfs repo provides public-feed starting data, but its maintainer warns that some trip times may be obsolete. Validate routes and timetables before calling results current.

Metro ridership: GMRC provides route/fare information and reports; I did not verify a public official station-by-hour ridership file. For PS-01/04/13, request operator data or label generated values as simulated.

Winning Back the Empty Seats

OFFLINE OPERATION: Not required

What the problem is really asking

Metro capacity is underused while roads remain congested. The operator needs to understand which non-riders could realistically switch from two-wheelers, buses/BRTS, or other modes, and what barrier prevents them from using the Metro.

Recommended solution architecture

Build a transparent commuter-conversion analysis rather than a generic ridership dashboard. Combine station-by-hour ridership, station access/last-mile options, fare comparisons, and a short survey (current mode, trip purpose, transfer/walk discomfort, fare sensitivity, service-time fit, and stated willingness to switch). Segment respondents by behaviour and barrier; rank segments by reachable population, likely conversion, intervention cost, and operational feasibility. Output three segments, one intervention for each, expected impact range, assumptions, and the evidence behind each recommendation.

Two-hour prototype scope

Use a small, clearly labelled synthetic survey plus station/time sample data if no real dataset is provided. Build a dashboard showing segment size, dominant barrier, score breakdown, and a proposal such as feeder shuttle, employer pass, secure bicycle/parking access, or integrated bus transfer. Do not claim causal lift from a survey alone; present it as a hypothesis to pilot.

Suggested stack

Python/pandas, scikit-learn clustering or rule-based segmentation, Plotly/Streamlit, GeoPandas for access zones. Start with interpretable scoring before using a black-box model.

GitHub / web resources

Existing projects / references: GTFS measures (US DOT Volpe) - shows how to derive transit frequency and estimated segment-level ridership from GTFS; it needs calibration data and is not a ready-made commuter-conversion model | Ahmedabad community GTFS feeds - AMTS, Janmarg/BRTS and Gandhinagar bus-feed starting point; the maintainer notes possible stale trip times for some data | GMRC reports & downloads - official project reports; do not assume these contain station-hour transaction data

Important gap / caveat

I did not verify a public, official Ahmedabad Metro station-by-hour ridership file during this research. The brief says participants must bring data or create realistic sample data. Use synthetic hourly ridership as a demo fixture and label it, or request actual AFC/ridership extracts from the organisers/operator.

Fix It Before It Breaks

OFFLINE OPERATION: Not required

What the problem is really asking

Faults in gates, vending machines, lifts and escalators are recorded after they occur. The operator wants a prioritized list of assets likely to fail in the next week and a practical night-shift maintenance roster.

Recommended solution architecture

Normalize fault logs into one row per incident with asset ID/type, station, fault code, first-seen time, downtime, fix, parts, repeat fault window, and maintenance cost. Start with a repeat-fault and severity baseline; then train a separate risk model by asset class if enough history exists. Rank by probability of repeat failure × operational impact × expected downtime, while accounting for how long the asset has been running since its last repair. Turn the ranked list into a technician-friendly roster with recommended inspection/checklist and reason for each priority.

Two-hour prototype scope

Upload a CSV, flag the top 10 assets, display confidence/risk band and explainable factors, and generate a night-shift checklist. Compare the model with a simple “most frequent faults” baseline. With few records, present a risk score—not an unvalidated probability of failure.

Suggested stack

Python/pandas, scikit-learn or XGBoost, SHAP/permutation importance for explanation, SQLite/PostgreSQL, Streamlit. Keep different asset classes separate because lifts and ticket gates fail for different reasons.

GitHub / web resources

Existing projects / references: Predictive maintenance CMAPSS pipeline - end-to-end ML pipeline using NASA turbofan run-to-failure sensors; useful algorithm and dashboard pattern, but its dataset is not metro equipment data | Great Expectations - can validate fault-log schemas, date ranges, missing asset IDs and duplicate incidents before modelling

Important gap / caveat

No verified repository was found that is ready to predict Ahmedabad Metro gate/lift/escalator failure from the operator’s maintenance logs. The CMAPSS project is a modelling reference only: fault-log fields and labels must match the metro assets.

The Technician in the Tunnel

OFFLINE OPERATION: REQUIRED

What the problem is really asking

Technicians need troubleshooting instructions while underground or in equipment rooms with no network. Long technical manuals are hard to search quickly, and the assistant must answer in Gujarati or English.

Recommended solution architecture

Package a fully local document assistant. Before the demo, extract text from manuals, OCR scanned pages, split text by section/page, create local embeddings and a searchable index, then run retrieval-augmented generation using a local language model. A technician types the fault or selects a photo/asset type; the system retrieves the best manual passages and returns numbered steps, required safety warnings, page/section citations, and a confidence indicator. For image input, treat visual fault recognition as a separate optional component; don't imply a text-only RAG model understands photographs.

Two-hour prototype scope

Pre-index two to five sample manuals before the event. Build one search/chat screen with source-page citations and a "no relevant procedure found" response. Test with known fault questions and completely disconnect the network. Cache all model weights and packages in advance.

Suggested stack

Local Ollama/Gemma or another locally runnable model, PyMuPDF/pypdf, Tesseract OCR for scanned PDFs, FAISS/Chroma/SQLite search, a small local UI. Use answer prompts that require evidence and allow refusal.

GitHub / web resources

Existing projects / references: offline_RAG_chatbot - local PDF RAG, multilingual embeddings, image extraction and offline voice components | xj16/localrag - offline document Q&A with citations and a fallback extractive answer path | Ollama - local model runtime; download model files before going offline

Important gap / caveat

These are general document assistants, not railway-maintenance-certified products. The manuals, terminology, Gujarati quality, OCR accuracy and safety procedure must be tested. Always show the source and retain human responsibility for safety-critical work.

No Day Is an Average Day

OFFLINE OPERATION: Not required

What the problem is really asking

A fixed headway based on a typical weekday can over- or under-serve demand on festival, exam, rain or cricket-match days. The control room wants next-day headway recommendations by time band, with an operator-readable reason.

Recommended solution architecture

Forecast station or corridor demand for 15/30-minute intervals, using historical ridership, weekday/holiday, school/exam calendar, event venue and end time, rainfall forecast, and recent service disruptions. Convert demand forecasts into a recommendation using train capacity, target occupancy, existing headway and fleet limits. The explanation should identify the driver (e.g. event egress overlaps evening peak) and show the comparison with a normal day. Include operational constraints: available trains, turn-back capacity and minimum safe headway.

Two-hour prototype scope

Build a scenario dashboard for “normal weekday”, “heavy rain”, and “event day”. Use a simple seasonal baseline or gradient boosting before complex neural forecasting. Recommend a small set of headway bands and show predicted demand, expected crowding band and reason. Keep the recommendation advisory; do not issue live train-control commands.

Suggested stack

Pandas, scikit-learn/LightGBM or Prophet, Open-Meteo weather forecast, event/holiday CSV, Plotly/Streamlit. For a demo with no true ridership history, use synthetic series and disclose that it is simulated.

GitHub / web resources

Existing projects / references: GTFS measures - can derive scheduled service frequency and transit segment measures from GTFS, but does not itself forecast event-driven headways | Open-Meteo forecast API - forecast variables such as precipitation and temperature that can be included as demand features | GTFS standard overview - standard schedule and real-time data concepts

Important gap / caveat

I did not find a verified, turnkey open-source Ahmedabad Metro headway recommender. It is a forecasting + operations-constraints problem; weather or event data by themselves do not establish passenger demand.

The Stadium Surge

OFFLINE OPERATION: Not required

What the problem is really asking

A match, Navratri night or riverfront event can push large crowds towards one or two stations in a short period. Operators need a minute-by-minute crowd management plan, including gates, holding areas, train additions and announcements in Gujarati, Hindi and English.

Recommended solution architecture

Convert event attendance into a time-distributed arrival/exit estimate, then estimate the share using Metro and distribute them among nearby stations and gates. Compare predicted flows with safe gate, corridor and platform capacities, train capacity, scheduled headway and walking time. Generate a timeline of actions and announcement scripts. If sample CCTV frames are available, crowd-counting can help estimate density; if not, use an explicit attendance curve and scenario slider. Simulation should support—not replace—operator and emergency-service judgment.

Two-hour prototype scope

Inputs: venue, attendance, expected Metro share, event end, selected station, gate/platform capacity, train capacity and baseline headway. Outputs: 1-minute/5-minute forecast curve, red/amber/green crowding, gates/holding plan and multilingual announcement templates. Provide low/medium/high attendance scenarios.

Suggested stack

Python simulation engine, SUMO pedestrian simulation for more advanced validation, a crowd-counting model such as CSRNet for images, and IndicTrans2 or pre-approved templates for translation.

GitHub / web resources

Existing projects / references: Eclipse SUMO pedestrian simulation - supports pedestrian-flow simulation and interactions with transport networks | CSRNet crowd counting implementation - density-map-based people counting for crowded images; needs suitable camera-domain evaluation | AI4Bharat IndicTrans2 - open-source translation models for 22 scheduled Indian languages, including Gujarati and Hindi

Important gap / caveat

No repository found supplies a validated, ready-to-run Ahmedabad event crowd plan with actual gate/platform capacities and train dispatch rules. Counts, arrival curves, capacities and station geometry are essential inputs; announcement translations should be checked by a fluent speaker.

Footfall to Revenue

OFFLINE OPERATION: Not required

What the problem is really asking

Vacant retail units and advertising panels are difficult to sell when the operator cannot quantify station footfall or explain why one location is worth more than another.

Recommended solution architecture

Build a station inventory with space dimensions, position, daily footfall, peak/off-peak profile, sightline/visibility, dwell-time proxy and nearby audience flow. Estimate potential views/impressions with a transparent formula and confidence interval, then create pricing bands and a one-page sales pitch for each available spot. Distinguish measured footfall from estimated ad impressions and avoid promising advertiser conversion. Use aggregate counts; avoid face identity and unnecessary demographic inference.

Two-hour prototype scope

For each vacant panel/shop, generate a PDF/HTML sales card showing station, location, peak/off-peak footfall, monthly estimated exposures, a pricing tier, the assumptions, and a suggested advertiser category. Include “data quality: measured/simulated” and a simple “what would improve confidence” section.

Suggested stack

People counting using YOLO + multi-object tracking, zone-based footfall aggregation, pandas rate-card calculator, and a Streamlit/React dashboard. If video analytics is not available, start with operator footfall counts.

GitHub / web resources

Existing projects / references: smart-retail-analytics - computer-vision platform for visitor counting, foot-traffic heatmaps, queue lengths and dashboards; not a metro advertising-pricing engine | Ultralytics YOLO documentation - object detection/tracking building blocks; evaluate accuracy on the actual station camera angle

Important gap / caveat

I did not locate a complete open-source metro ad-rate-card generator. The main value is the evidence and pricing methodology, not simply detecting people. Actual pricing needs viewing geometry, occupancy/footfall data, comparable rates and commercial validation.

Every Complaint Heard

OFFLINE OPERATION: REQUIRED

What the problem is really asking

Complaints arrive through multiple channels in Gujarati, Hindi, English and mixed Gujlish. Urgent safety issues can be buried in a backlog; a controller must route complaints by department, assign urgency and draft a response while offline.

Recommended solution architecture

Create an offline intake and triage queue. Normalize text and attachments, detect/record language, classify department (safety, cleanliness, fare, lift, staff behaviour, etc.), and compute a transparent urgency score. Safety, stranded passenger, accessibility and service-blocking reports should trigger deterministic rules that are easy to audit. Use a multilingual classifier for suggestions; draft replies from approved templates/local generation. Store records and attachments locally, then sync an outbox with duplicate IDs when connectivity returns. Keep an audit trail for edits, assignments and acknowledgement.

Two-hour prototype scope

Seed 30–100 labelled sample complaints, including code-switched Gujlish and spelling variants. Show priority, predicted department, reason, draft response in the complainant's language, offline persistence and a sync-pending badge. Add a "human override" control. Make the safety escalation rules visible.

Suggested stack

Local SQLite, Python/FastAPI or a local-first PWA, IndicBERT-family classifier, rules for urgent phrases, and preapproved multilingual reply templates. IndicTrans2 is a translation option if the model can be preloaded and run on the available hardware.

GitHub / web resources

Existing projects / references: AI4Bharat IndicBERT - pretrained model includes Gujarati and Hindi among 12 Indian languages; classification still requires task-specific fine-tuning or labelled examples | AI4Bharat IndicTrans2 - translation models for 22 scheduled Indian languages, including Gujarati and Hindi | offline_RAG_chatbot - reference for local UI/model packaging and offline execution, although it is a document-Q&A project rather than complaint triage

Important gap / caveat

No verified public repo was found that exactly performs offline Gujarati/Hindi/English/Gujlish complaint triage for a metro operator. Code-switching and transliterated Gujarati are the hardest edge cases; you need labelled examples and a clear, tested safety escalation policy.

One City, One Journey

OFFLINE OPERATION: Not required

What the problem is really asking

Metro, BRTS and AMTS are operated separately. A passenger needs a single itinerary showing each leg, transfer walking time and an understandable total fare.

Recommended solution architecture

Represent each operator using a shared stop/route/trip model, find a multimodal path, and calculate a total-cost estimate with an explicit fare rule per leg. Use walking paths between nearby stations/stops to create transfer edges. Show alternatives with total travel time, wait time, walking distance, number of transfers and data freshness. Fare splitting between operators should be proposed as a separate ledger rule; avoid implying that separate tickets have become a single payment credential.

Two-hour prototype scope

For a limited sample OD pair, display 2–3 possible itineraries and detailed legs. Use community Ahmedabad AMTS/BRTS GTFS as an initial feed, build a small Metro GTFS from verified station/timetable inputs, and use OpenTripPlanner if setup allows. If setup is too heavy for the time limit, use a small graph for the demo but preserve GTFS-style data structures.

Suggested stack

GTFS Static data, OpenTripPlanner, OpenStreetMap walking network, custom fare calculator and MapLibre/Leaflet UI.

GitHub / web resources

Existing projects / references: OpenTripPlanner - open-source multimodal journey planner combining GTFS and OpenStreetMap with routing APIs | GTFS standard overview - schedule/realtime data specification; includes stops, routes, trips and optional fare/transfer files | Ahmedabad community GTFS feeds - AMTS, Janmarg/BRTS and GTSL feeds assembled from web sources; validate before use | GMRC route and fares - official source to cross-check Metro stations and fares

Important gap / caveat

The community GTFS repository does not itself provide a confirmed official GMRC Metro GTFS feed or a unified fare account. Timetable correctness and transfer paths must be checked; fare-splitting is business logic you would need to implement and explain.

Clear the Exit

OFFLINE OPERATION: REQUIRED

What the problem is really asking

Auto-rickshaw clusters, illegal parking and encroachments choke station exits. Field enforcement teams need to capture evidence without network access and aggregate repeated incidents into a daily hotspot map.

Recommended solution architecture

Make the mobile reporting path local-first. Capture a photo, GPS position, station/exit, timestamp, issue category, severity, evidence note and optional confidence score. Use an on-device model only as a suggestion unless it has been trained on the right classes; allow the field officer to correct the result. Store records and images locally, then sync later. The back-office view clusters repeated reports by road segment, exit and time, and ranks hotspots by frequency, safety/accessibility impact and persistence.

Two-hour prototype scope

Ship an offline form and map first, then add one on-device model that suggests vehicle/obstruction categories. Preload station exit locations and basemap tiles. Demonstrate airplane mode, save several reports, generate a hotspot map after reconnecting or from the local database, and show the image with each report.

Suggested stack

Android/PWA local SQLite or IndexedDB, GPS and camera, YOLO/ONNX/TFLite for image suggestions, GeoJSON/Leaflet or QGIS/QField for spatial reporting.

GitHub / web resources

Existing projects / references: Ultralytics YOLO docs - real-time object detection/segmentation and tracking building blocks; custom issue classes require task-specific data | QField offline workflow - field data capture with packaged local layers and later synchronization | QField documentation - mobile GIS field collection and project packaging

Important gap / caveat

No verified end-to-end repository was found specifically for offline Ahmedabad Metro-exit enforcement. A generic object detector will not reliably infer “illegal parking” from object category alone; use context, a trained classifier and officer confirmation.

Diversions Without Disruption

OFFLINE OPERATION: Not required

What the problem is really asking

Roadwork and Metro construction change road access and bus routes. Advisories are often late or single-language, and passengers learn about the diversion after reaching the blocked section.

Recommended solution architecture

Model each work zone as a georeferenced road segment/polygon with start/end time, closure type and affected modes. Intersect it with bus route shapes/stops and walking transfers to identify affected routes and stations. Generate an alternate-route map plus channel-specific messages: full advisory, 160–300-character WhatsApp-ready text, station notice and multilingual version. Include validity times, stop IDs, alternative boarding points and who approved the notice. An operator must verify each proposed diversion before publication.

Two-hour prototype scope

Upload a construction schedule CSV and draw/choose a closure on the map; automatically list intersecting bus routes and affected stops; generate English/Gujarati/Hindi advisory drafts and export to a map image and text files. Use a small Ahmedabad route sample if no verified real-time feed exists.

Suggested stack

GTFS routes/stops/shapes, OpenStreetMap road graph, GeoPandas/Shapely spatial intersection, OpenTripPlanner or OSRM for alternate path ideas, IndicTrans2/templates for multilingual messages, MapLibre/Leaflet.

GitHub / web resources

Existing projects / references: OpenTripPlanner - multimodal routing and disruption-aware itinerary foundation; works from GTFS and OpenStreetMap | GTFS standard overview - common transit feed format with schedule and real-time concepts including service alerts | Ahmedabad community GTFS - potential AMTS/BRTS route starting data, requiring freshness checks | IndicTrans2 - Gujarati/Hindi/English translation support

Important gap / caveat

I did not find a ready-made repository that consumes Ahmedabad construction schedules and safely publishes verified bus diversions. The difficult part is current work-zone data and operations approval, not message generation alone.

Beat the Heat

OFFLINE OPERATION: Not required

What the problem is really asking

High summer heat discourages walking to/from stations. The city needs to suggest walking routes with shade and identify exposed stretches where shelters or tree cover would give the greatest benefit.

Recommended solution architecture

Build a walking graph and assign every edge a shade score based on tree canopy, building shade at the requested time/date, and the route's sun exposure. Compute a route that trades extra walking time for shade, with a slider between "fastest" and "coolest". Separately rank candidate improvements by exposed pedestrian volume, shade deficit, proximity to station, length and feasibility. Show the evidence layer so that missing tree/building data is visible.

Two-hour prototype scope

Select origin and a Metro station; display fastest and shade-prioritized paths with total distance, estimated walk time and percentage of segments estimated in shade. Then highlight the worst exposed segments on the map. If building heights or tree data are missing, mark the estimate as incomplete instead of inventing shade.

Suggested stack

OpenStreetMap/OSMnx pedestrian graph, building footprints and tree inventory, solar-position calculation, GeoPandas/Shapely and a Dijkstra/A* weighted route engine.

GitHub / web resources

Existing projects / references: shade-route (NYC) - directly relevant application: shade-weighted walking routes using building-shadow simulation, tree canopy and custom graph routing | Schattenrouten - browser route system estimating building/tree shadows from OSM features and sun position

Important gap / caveat

These are excellent algorithmic references but are not Ahmedabad-ready: the first is configured around New York data and both need local, accurate building/tree inputs. OpenStreetMap canopy/building-height completeness may be insufficient for block-level comfort estimates.

Monsoon Mode

OFFLINE OPERATION: Not required

What the problem is really asking

Heavy rain and waterlogging disrupt station entrances, approach roads and parking areas. Operators want a station-specific 24-hour readiness plan, including possible entrance closures, pump placement and commuter communication.

Recommended solution architecture

Combine forecast rainfall intensity/duration with terrain, drainage connectivity, surface type, past waterlogging reports and station entrance elevations. Compute a screening risk score for each entrance and approach road, then generate readiness actions with trigger thresholds and uncertainty. Separate “forecast rainfall” from “predicted local water depth”: the second requires drainage capacity, blockage and observed flood calibration. Use a map showing risk plus the evidence used for each station action.

Two-hour prototype scope

For 3–5 sample stations, build low/medium/high risk scenarios driven by rainfall and known low points. Produce a checklist: inspect drains, place pumps at verified locations, protect electrical rooms, stage staff, prepare alternate entrance/route messaging. Label simulated drainage and thresholds explicitly. Do not use the prototype as an authority for real entrance closures.

Suggested stack

Open-Meteo precipitation forecast, elevation raster (Copernicus DEM or another available DEM), OSM road/drainage features, QGIS/GeoPandas/rasterio, simple runoff/risk scoring, dashboard and alert templates.

GitHub / web resources

Existing projects / references: FlowState urban-flood nowcasting (SIH prototype) - combines rainfall, runoff/drainage modelling, ML, risk maps and safe-route outputs; repository explicitly describes itself as a prototype, not a validated operational system | RAKSHak urban flood nowcasting (SIH prototype) - another Indian hackathon-style flood-risk and civic-response reference | Open-Meteo API docs - precipitation and weather variables for forecast inputs | Copernicus DEM GLO-90 / GLO-30 information - global elevation data; note access conditions differ by resolution/user category

Important gap / caveat

Both flood repositories are prototypes, not validated municipal warning systems. DEM resolution is not enough to predict street-level water depth by itself, and crowdsourced drainage mapping may be incomplete. Real recommendations need city drainage information, verified historical flood points and operator review.

The Two-Minute Morning Brief

OFFLINE OPERATION: Not required

What the problem is really asking

Daily MIS reports require manual merging of Metro, BRTS and AMTS data. The government needs one consistent brief that highlights exceptions and suggests likely causes without taking hours to prepare.

Recommended solution architecture

Create a repeatable ETL pipeline: ingest source files, standardize dates/agency/station/route IDs, validate schemas and totals, calculate day-over-day and same-weekday changes, and detect missing records or unusual trends. Generate a one-page brief with headline KPIs, red/amber exceptions, short cause hypotheses and a traceable source link for every statement. Use rule-based explanations first ("ridership down 12%; a gate outage and high rainfall are recorded for the same interval"). If an LLM drafts prose, constrain it to observed facts and label hypotheses as hypotheses.

Two-hour prototype scope

Drop in three CSV files and produce an HTML/PDF one-pager: ridership, fare revenue, trips/operations, reliability, complaints, top anomalies and data-quality warnings. Include a downloadable exceptions table and show which source rows triggered each flag. Compare against same weekday in the prior 4 weeks instead of comparing every day to the immediately previous day.

Suggested stack

Python/pandas, Great Expectations or dataframe assertions, robust z-score/seasonal baseline, Jinja/HTML-to-PDF or ReportLab, Streamlit and optional local LLM for explanation text.

GitHub / web resources

Existing projects / references: Great Expectations GitHub - open-source data quality expectations and validation output; checks inputs, but does not build the MIS report for you | Great Expectations docs - examples for validating recurring tabular datasets | GTFS standard overview - useful context for normalizing transit route/schedule concepts when source schemas differ

Important gap / caveat

This is one of the best two-hour prototypes because the data flow is focused and demonstrable. Do not let the language model manufacture causes: each cause must link to an independently observed outage, weather record or operations note.

Which problem should you choose?

A good two-hour submission has one complete user journey, meaningful technical work, and a clear demo story. The best choice depends on whether your team already has local models, sample data and mapping/runtime setup ready.

Choice	Best reason to choose it	Primary risk	Demo you can finish
PS-13: Morning brief	Highest chance of polishing an end-to-end workflow quickly	Dirty or inconsistent source CSVs	Upload 3 files → validate → flag anomalies → produce one-page report
PS-07: Complaint triage	Strong offline AI + clear operational impact	Gujlish labels/model setup and safe urgency logic	Import batch → route/score → draft local-language reply → queue offline
PS-03: Technician assistant	Clear value and technical depth with local RAG	Manual parsing and local model readiness	Ask known fault → cited steps/page → airplane-mode demo
PS-08: One city, one journey	Most visual public-facing demo and reusable routing stack	Feed validation and OTP setup time	Choose origin/destination → 2 itineraries → fare/transfer details
PS-09: Clear the exit	Visual AI plus offline-first GIS field workflow	Custom classes and geolocation quality	Capture photo → save offline → hotspot map
PS-06: Footfall to revenue	Business-facing output and easy-to-explain ROI story	Footfall evidence and pricing assumptions	Choose vacant panel → footfall evidence → rate card + pitch

Final recommendation

For the safest route to a polished two-hour demo, choose PS-13. For an AI-forward problem with an explicit offline requirement, choose PS-07 if you can pre-load and test the model, or PS-03 if your manuals are searchable and the local RAG stack already runs. Choose PS-08 when you have time to validate the transit feeds and set up the routing engine. For PS-12, be especially clear that the result is a preparedness prototype—not a validated flood prediction or authority to close an entrance.

Sources reviewed and verification notes

- The user-provided Metro Hacks Participant Brief was the source of the 13 PS descriptions, expected deliverables and offline labels.
- External research focused on public GitHub repositories and official project/standard documentation: OpenTripPlanner, GTFS, AI4Bharat, YOLO, QField, Open-Meteo, Copernicus DEM, SUMO, Great Expectations and selected application repositories.
- Repository README/documentation text was reviewed online, but source code was not cloned, installed or executed. A live repository page is not proof of production readiness, model accuracy, maintenance quality or compatibility with a team laptop.
- Check license, model-weight permissions, data license, privacy implications, dependencies and current commit activity before copying code into a real deployment.